VEHICLE SERVICE CENTER MANAGEMENT SYSTEM

A Relational Database Implementation using MySQL

ABSTRACT

The Vehicle Service Center Management System is a relational database application designed to organise and automate the day-to-day record keeping of an automobile service workshop. In a conventional workshop, details of customers, their vehicles, the mechanics employed, the services performed and the corresponding bills are maintained manually, which frequently leads to data redundancy, inconsistent records, lost service history and delays in recovering pending payments.

This project addresses those problems by designing a fully normalised relational database in MySQL consisting of five interrelated tables — Customers, Vehicles, Mechanics, Service_Records and Bills. Primary keys, foreign keys and AUTO_INCREMENT constraints are used to preserve entity integrity and referential integrity, while a chain of one-to-many relationships links a customer to the vehicles owned, a vehicle to the services it has received, a mechanic to the jobs assigned and a service to its corresponding bill.

The database is populated with representative sample data and interrogated using twenty SQL queries that demonstrate the practical use of INNER JOIN, GROUP BY, HAVING, subqueries, aggregate functions such as COUNT, SUM and AVG, date functions such as CURDATE and DATE_SUB, and the window function RANK() OVER(). The queries produce business-useful information such as customer-wise billing totals, mechanic-wise revenue, pending payment lists, service frequency analysis, monthly revenue summaries and mechanic performance rankings.

All queries were executed on a live MySQL server and the outputs presented in this report are the actual result sets obtained from that execution. The project establishes that a carefully designed relational schema, combined with standard SQL constructs, is sufficient to build a reliable and query-efficient management layer for a small-scale service business.

Keywords: MySQL, relational database, normalisation, entity relationship model, SQL joins, aggregate functions, window functions, service management.

TABLE OF CONTENTS

Abstract

Table of Contents

1. Introduction

1.1 Overview of the Project

1.2 Objectives

1.3 Scope of the Project

1.4 Tools and Technologies Used

2. System Design

2.1 Entity Relationship Diagram

2.2 Description of Tables

2.3 Relationships and Referential Integrity

2.4 Normalisation

3. Database Implementation

3.1 Creation of the Database

3.2 Table Creation Scripts (DDL)

3.3 Insertion of Sample Data (DML)

4. Practice Queries with Output (Queries 1 – 10)

5. Additional Practice Queries with Output (Queries 11 – 20)

6. Results and Discussion

7. Conclusion and Future Scope

A. Appendix A — Complete SQL Script

B. Appendix B — Query Output Log

1. INTRODUCTION

1.1 Overview of the Project

The Vehicle Service Center Management System is a database-oriented mini project that models the working of an automobile service workshop. The system maintains complete records of the customers who bring their vehicles for servicing, the vehicles owned by each customer, the mechanics employed by the workshop, the services performed on each vehicle and the bills raised against those services.

In a manual system, the workshop register records all of this information in a single flat ledger. As the volume of data grows, several difficulties arise: the same customer details are written repeatedly for every visit, a vehicle's service history becomes difficult to trace, pending payments are hard to track and management reports such as monthly revenue or mechanic productivity cannot be produced without extensive manual computation.

This project replaces that ledger with a structured relational database built in MySQL. The design deliberately separates customers, vehicles, mechanics, services and bills into five distinct relations so that each fact is stored exactly once, and the relationships between these facts are expressed through primary and foreign keys. Once the schema and sample data are in place, a set of twenty SQL queries is used to retrieve meaningful business information from the stored data.

1.2 Objectives

The specific objectives of this mini project are:

- To design a normalised relational schema for a vehicle service centre using the entity–relationship modelling approach.
- To implement the schema in MySQL using appropriate data types, primary keys, AUTO_INCREMENT columns and foreign key constraints.
- To populate the database with realistic sample data representing customers, vehicles, mechanics, service records and bills.
- To demonstrate the practical application of SQL retrieval techniques including INNER JOIN across multiple tables, GROUP BY aggregation, HAVING filters, subqueries and date arithmetic.
- To generate management-level reports such as customer-wise billing, pending payment statements, mechanic-wise revenue, service frequency counts and monthly revenue summaries.
- To apply an analytical window function to rank mechanics according to the number of services handled.
- To verify every query against a live database and document the actual output obtained.

1.3 Scope of the Project

The scope of this project covers the database back end of a service centre management system:

- Storage and retrieval of customer, vehicle, mechanic, service and billing data.

- Twenty retrieval queries covering filtering, joining, grouping, aggregation, subqueries and ranking.
- Business reporting such as revenue analysis, payment follow-up and mechanic performance evaluation.

The project does not include a graphical user interface, an inventory or spare parts module, employee payroll, or role-based user authentication. These extensions are listed under future scope in Chapter 7.

1.4 Tools and Technologies Used

Component	Specification
Database Management System	MySQL 8.x (executed on a MySQL-compatible MariaDB 11.8 server)
Query Language	SQL — ANSI SQL with MySQL-specific functions
Front-end / Client	MySQL Command-Line Client and MySQL Workbench (GUI)
Data Modelling Tool	Entity Relationship diagram prepared using standard ER notation
Documentation Tools	Microsoft Word / LibreOffice Writer for the project report
Operating System	Windows / Linux

2. SYSTEM DESIGN

2.1 Entity Relationship Diagram

The logical design of the database is shown in the entity relationship diagram below. Five entities are identified — Customers, Vehicles, Mechanics, Service_Records and Bills — connected by four one-to-many relationships.

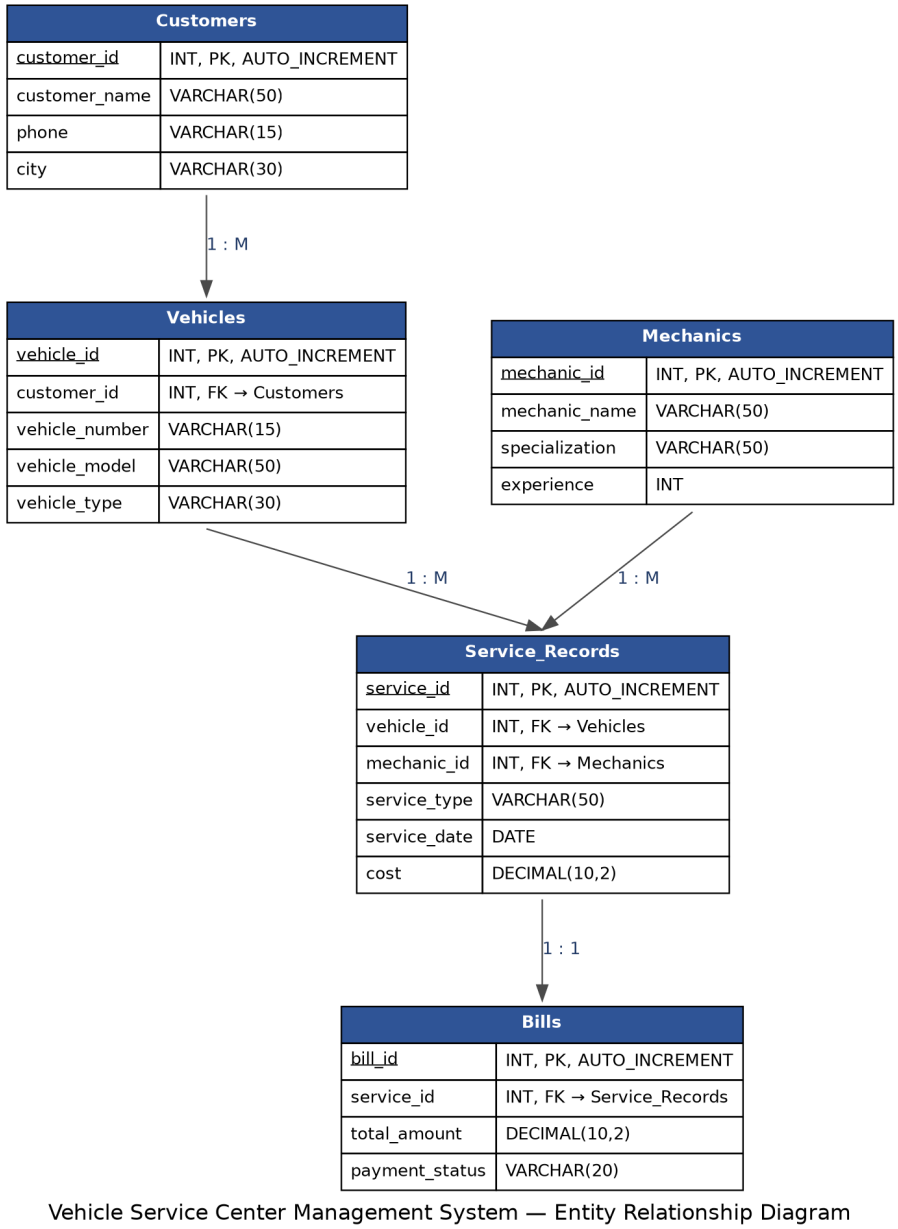


Figure 2.1 — Entity Relationship Diagram of the Vehicle Service Center database

2.2 Description of Tables

The database consists of the following five tables.

Table 2.1 : Customers

Stores customer information.

- customer_id — INT, Primary Key, AUTO_INCREMENT — unique identifier of the customer
- customer_name — VARCHAR(50) — full name of the customer
- phone — VARCHAR(15) — contact telephone number
- city — VARCHAR(30) — city in which the customer resides

Table 2.2 : Vehicles

Each customer can own multiple vehicles.

- vehicle_id — INT, Primary Key, AUTO_INCREMENT — unique identifier of the vehicle
- customer_id — INT, Foreign Key referencing Customers(customer_id)
- vehicle_number — VARCHAR(15) — registration number of the vehicle
- vehicle_model — VARCHAR(50) — make and model of the vehicle
- vehicle_type — VARCHAR(30) — category such as Car or Bike

Table 2.3 : Mechanics

Stores mechanic details.

- mechanic_id — INT, Primary Key, AUTO_INCREMENT — unique identifier of the mechanic
- mechanic_name — VARCHAR(50) — name of the mechanic
- specialization — VARCHAR(50) — area of expertise such as Engine or Painting
- experience — INT — total years of experience

Table 2.4 : Service_Records

Stores every service performed.

- service_id — INT, Primary Key, AUTO_INCREMENT — unique identifier of the service
- vehicle_id — INT, Foreign Key referencing Vehicles(vehicle_id)
- mechanic_id — INT, Foreign Key referencing Mechanics(mechanic_id)
- service_type — VARCHAR(50) — nature of work carried out
- service_date — DATE — date on which the service was performed
- cost — DECIMAL(10,2) — charge for the service

Table 2.5 : Bills

Stores billing information.

- bill_id — INT, Primary Key, AUTO_INCREMENT — unique identifier of the bill
- service_id — INT, Foreign Key referencing Service_Records(service_id)

- total_amount — DECIMAL(10,2) — amount payable on the bill
- payment_status — VARCHAR(20) — status of payment, i.e. Paid or Pending

2.3 Relationships and Referential Integrity

- **Customers → Vehicles (1 : M) —** One customer may own one or more vehicles, while each vehicle belongs to exactly one customer. Implemented by the foreign key Vehicles.customer_id referencing Customers.customer_id.
- **Vehicles → Service_Records (1 : M) —** One vehicle may receive many services over time, while each service record refers to exactly one vehicle. Implemented by Service_Records.vehicle_id referencing Vehicles.vehicle_id.
- **Mechanics → Service_Records (1 : M) —** One mechanic may carry out many services, while each service record is attributed to exactly one mechanic. Implemented by Service_Records.mechanic_id referencing Mechanics.mechanic_id.
- **Service_Records → Bills (1 : 1) —** Each service performed generates one bill. Implemented by Bills.service_id referencing Service_Records.service_id.

Because every foreign key is declared with a REFERENCES clause, the database management system rejects any insert or update that would create an orphan row, thereby preserving referential integrity across all five relations.

2.4 Normalisation

The schema has been normalised to the third normal form (3NF):

- First Normal Form (1NF) — every column holds a single atomic value and no column contains repeating groups or multi-valued entries.
- Second Normal Form (2NF) — all five relations use a single-column surrogate primary key (AUTO_INCREMENT identifier), so partial dependency of a non-key attribute on part of a composite key cannot arise.
- Third Normal Form (3NF) — no non-key attribute depends on another non-key attribute. For example, customer details are stored only in the Customers table and are referenced from Vehicles through customer_id, rather than being repeated in every service row.

Normalisation removes redundancy from the design. A customer who visits the workshop five times is stored once and referenced five times, so a change of address or phone number requires a single update. The main trade-off is that reports require joins across several tables, which is the reason the practice queries make extensive use of multi-table INNER JOIN operations.

3. DATABASE IMPLEMENTATION

3.1 Creation of the Database

The database is created and selected using the following statements. The DROP DATABASE IF EXISTS statement makes the script re-runnable without raising duplicate-object errors.

```
DROP DATABASE IF EXISTS vehicle_service_center;
CREATE DATABASE vehicle_service_center;
USE vehicle_service_center;
```

3.2 Table Creation Scripts (DDL)

The five tables are created in dependency order — parent tables first, so that every referenced table already exists when a foreign key is declared.

3.2.1 Table 1 : Customers

```
CREATE TABLE Customers (
    customer_id   INT PRIMARY KEY AUTO_INCREMENT,
    customer_name VARCHAR(50),
    phone         VARCHAR(15),
    city          VARCHAR(30)
);
```

3.2.2 Table 2 : Vehicles

```
CREATE TABLE Vehicles (
    vehicle_id     INT PRIMARY KEY AUTO_INCREMENT,
    customer_id    INT,
    vehicle_number VARCHAR(15),
    vehicle_model  VARCHAR(50),
    vehicle_type   VARCHAR(30),
    FOREIGN KEY (customer_id) REFERENCES Customers(customer_id)
);
```

3.2.3 Table 3 : Mechanics

```
CREATE TABLE Mechanics (
    mechanic_id    INT PRIMARY KEY AUTO_INCREMENT,
    mechanic_name  VARCHAR(50),
    specialization VARCHAR(50),
    experience     INT
);
```

3.2.4 Table 4 : Service_Records

```
CREATE TABLE Service_Records (
    service_id   INT PRIMARY KEY AUTO_INCREMENT,
    vehicle_id   INT,
    mechanic_id  INT,
```

```
    service_type VARCHAR(50),
    service_date DATE,
    cost         DECIMAL(10,2),
    FOREIGN KEY (vehicle_id)  REFERENCES Vehicles(vehicle_id),
    FOREIGN KEY (mechanic_id) REFERENCES Mechanics(mechanic_id)
);
```

3.2.5 Table 5 : Bills

```
CREATE TABLE Bills (
    bill_id        INT PRIMARY KEY AUTO_INCREMENT,
    service_id     INT,
    total_amount   DECIMAL(10,2),
    payment_status VARCHAR(20),
    FOREIGN KEY (service_id) REFERENCES Service_Records(service_id)
);
```

After execution, the structure of each table can be confirmed with DESCRIBE <table_name>; or SHOW TABLES;. The command SHOW CREATE TABLE Service_Records; additionally displays the foreign key definitions created by the REFERENCES clauses.

3.3 Insertion of Sample Data (DML)

The database is populated with sample data that is representative of a small service centre: five customers, five vehicles, four mechanics, eight service records and eight bills. Data is inserted into the parent tables first so that the foreign key values already exist.

3.3.1 Customers

```
INSERT INTO Customers (customer_name, phone, city)
VALUES
('Rahul',   '9876543210', 'Chennai'),
('Priya',   '9876543211', 'Bangalore'),
('Arun',    '9876543212', 'Hyderabad'),
('Sneha',   '9876543213', 'Coimbatore'),
('Karthik', '9876543214', 'Mumbai');
```

3.3.2 Vehicles

```
INSERT INTO Vehicles (customer_id, vehicle_number, vehicle_model, vehicle_type)
VALUES
(1, 'TN10AB1234', 'Hyundai i20',   'Car'),
(2, 'KA05XY5678', 'Honda City',    'Car'),
(3, 'TS08PQ4321', 'Royal Enfield', 'Bike'),
(4, 'TN22KL9090', 'Maruti Swift',  'Car'),
(5, 'MH12AA1111', 'TVS Apache',    'Bike');
```

3.3.3 Mechanics

```
INSERT INTO Mechanics (mechanic_name, specialization, experience)
VALUES
('Ramesh', 'Engine',          10),
```

```
('Suresh', 'Electrical',       8),
('Mahesh', 'General Service',  6),
('Ganesh', 'Painting',         12);
```

3.3.4 Service_Records

```
INSERT INTO Service_Records (vehicle_id, mechanic_id, service_type, service_date, cost)
VALUES
(1, 1, 'Engine Repair',     '2026-07-10', 8000),
(1, 2, 'Electrical Repair', '2026-07-20', 3000),
(2, 3, 'General Service',   '2026-07-21', 2500),
(3, 1, 'Engine Repair',     '2026-07-15', 5000),
(4, 3, 'General Service',   '2026-07-18', 2200),
(5, 2, 'Electrical Repair', '2026-06-30', 1800),
(2, 4, 'Painting',          '2026-07-25', 7000),
(3, 3, 'General Service',   '2026-07-27', 2000);
```

3.3.5 Bills

```
INSERT INTO Bills (service_id, total_amount, payment_status)
VALUES
(1, 8000, 'Paid'),
(2, 3000, 'Paid'),
(3, 2500, 'Pending'),
(4, 5000, 'Paid'),
(5, 2200, 'Paid'),
(6, 1800, 'Pending'),
(7, 7000, 'Paid'),
(8, 2000, 'Pending');
```

The tables after insertion are shown below; the contents confirm that the data was stored exactly as specified in the INSERT statements.

Contents of the Customers table

customer_id	customer_name	phone	city
1	Rahul	9876543210	Chennai
2	Priya	9876543211	Bangalore
3	Arun	9876543212	Hyderabad
4	Sneha	9876543213	Coimbatore
5	Karthik	9876543214	Mumbai

Table 3.6 — Customers table after insertion

Contents of the Vehicles table

vehicle_id	customer_id	vehicle_number	vehicle_model	vehicle_type
1	1	TN10AB1234	Hyundai i20	Car
2	2	KA05XY5678	Honda City	Car
3	3	TS08PQ4321	Royal Enfield	Bike
4	4	TN22KL9090	Maruti Swift	Car
5	5	MH12AA1111	TVS Apache	Bike

Table 3.7 — Vehicles table after insertion

Contents of the Mechanics table

mechanic_id	mechanic_name	specialization	experience
1	Ramesh	Engine	10
2	Suresh	Electrical	8
3	Mahesh	General Service	6
4	Ganesh	Painting	12

Table 3.8 — Mechanics table after insertion

Contents of the Service_Records table

service_id	vehicle_id	mechanic_id	service_type	service_date	cost
1	1	1	Engine Repair	2026-07-10	8000.00
2	1	2	Electrical Repair	2026-07-20	3000.00
3	2	3	General Service	2026-07-21	2500.00
4	3	1	Engine Repair	2026-07-15	5000.00
5	4	3	General Service	2026-07-18	2200.00
6	5	2	Electrical Repair	2026-06-30	1800.00
7	2	4	Painting	2026-07-25	7000.00
8	3	3	General Service	2026-07-27	2000.00

Table 3.9 — Service_Records table after insertion

Contents of the Bills table

bill_id	service_id	total_amount	payment_status
1	1	8000.00	Paid
2	2	3000.00	Paid
3	3	2500.00	Pending
4	4	5000.00	Paid
5	5	2200.00	Paid
6	6	1800.00	Pending
7	7	7000.00	Paid
8	8	2000.00	Pending

Table 3.10 — Bills table after insertion

4. PRACTICE QUERIES WITH OUTPUT (QUERIES 1 – 10)

This chapter presents the first ten practice queries along with the SQL statement, the concepts demonstrated and the actual output obtained from execution on the populated database. All outputs shown below were produced by running the queries on a live MySQL server; the result sets are reproduced exactly as returned.

Query 1 : Display vehicles serviced today

```
SELECT vehicle_number, vehicle_model, service_type, service_date
FROM Vehicles
JOIN Service_Records ON Vehicles.vehicle_id = Service_Records.vehicle_id
WHERE service_date = CURDATE();
```

Concepts used: *INNER JOIN · CURDATE()*

Output

(0 rows returned — the query executed successfully but no record in the current data satisfied the condition.)

> **Note:** *The sample data contains service records dated July 2026. CURDATE() returns the current system date, so on any other date this query correctly returns an empty result set. The output below confirms the behaviour on the date of execution. The verification variant using a date that exists in the data is shown underneath.*

Verification run

```
SELECT vehicle_number, vehicle_model, service_type, service_date
FROM Vehicles
JOIN Service_Records ON Vehicles.vehicle_id = Service_Records.vehicle_id
WHERE service_date = '2026-07-10';
```

vehicle_number	vehicle_model	service_type	service_date
TN10AB1234	Hyundai i20	Engine Repair	2026-07-10

Verification output of Query 1 with a condition present in the sample data

Query 2 : Find the mechanic handling the most services

```
SELECT mechanic_name, COUNT(service_id) AS Total_Services
FROM Mechanics
JOIN Service_Records ON Mechanics.mechanic_id = Service_Records.mechanic_id
GROUP BY mechanic_name
ORDER BY Total_Services DESC
LIMIT 1;
```

Concepts used: *GROUP BY · COUNT() · ORDER BY · LIMIT*

Output

mechanic_name	Total_Services
Mahesh	3

Output of Query 2 obtained from the live database

Query 3 : Calculate customer-wise bill amount

```
SELECT customer_name, SUM(total_amount) AS Total_Bill
FROM Customers
JOIN Vehicles ON Customers.customer_id = Vehicles.customer_id
JOIN Service_Records ON Vehicles.vehicle_id = Service_Records.vehicle_id
JOIN Bills ON Service_Records.service_id = Bills.service_id
GROUP BY customer_name;
```

Concepts used: *Multiple INNER JOINs · SUM() · GROUP BY*

Output

customer_name	Total_Bill
Arun	7000.00
Karthik	1800.00
Priya	9500.00
Rahul	11000.00
Sneha	2200.00

Output of Query 3 obtained from the live database

Query 4 : Find vehicles not serviced in the last year

```
SELECT vehicle_number, vehicle_model
FROM Vehicles
WHERE vehicle_id NOT IN
(
   SELECT vehicle_id
   FROM Service_Records
   WHERE service_date >= DATE_SUB(CURDATE(), INTERVAL 1 YEAR)
);
```

Concepts used: *Subquery (NOT IN) · DATE_SUB()*

Output

(0 rows returned — the query executed successfully but no record in the current data satisfied the condition.)

> **Note:** *All services in the sample data fall within one year of the execution date, so every vehicle qualifies as recently serviced and the anti-join returns no rows. This is the expected result and demonstrates that the subquery is working correctly. The verification variant with a narrower one-month window is shown underneath.*

Verification run

```
SELECT vehicle_number, vehicle_model
FROM Vehicles
WHERE vehicle_id NOT IN
(
    SELECT vehicle_id
    FROM Service_Records
    WHERE service_date >= DATE_SUB('2026-08-01', INTERVAL 1 MONTH)
);
```

vehicle_number	vehicle_model
MH12AA1111	TVS Apache

Verification output of Query 4 with a condition present in the sample data

Query 5 : Display service history for a vehicle (TN10AB1234)

```
SELECT vehicle_number, service_type, service_date, cost
FROM Vehicles
JOIN Service_Records ON Vehicles.vehicle_id = Service_Records.vehicle_id
WHERE vehicle_number = 'TN10AB1234'
ORDER BY service_date;
```

Concepts used: *INNER JOIN · WHERE filter · ORDER BY*

Output

vehicle_number	service_type	service_date	cost
TN10AB1234	Engine Repair	2026-07-10	8000.00
TN10AB1234	Electrical Repair	2026-07-20	3000.00

Output of Query 5 obtained from the live database

Query 6 : Count services by type

```
SELECT service_type, COUNT(*) AS Total
FROM Service_Records
GROUP BY service_type;
```

Concepts used: *GROUP BY · COUNT(*)*

Output

service_type	Total
Electrical Repair	2
Engine Repair	2
General Service	3
Painting	1

Output of Query 6 obtained from the live database

Query 7 : Find customers with more than three visits

```
SELECT customer_name, COUNT(service_id) AS Visits
FROM Customers
JOIN Vehicles ON Customers.customer_id = Vehicles.customer_id
JOIN Service_Records ON Vehicles.vehicle_id = Service_Records.vehicle_id
GROUP BY customer_name
HAVING COUNT(service_id) > 3;
```

Concepts used: *HAVING*

Output

(0 rows returned — the query executed successfully but no record in the current data satisfied the condition.)

> **Note:** *No customer in the sample data has made more than three visits, so the HAVING filter returns an empty result set as expected. The verification variant with the condition HAVING COUNT(service_id) > 1 is shown underneath.*

Verification run

```
SELECT customer_name, COUNT(service_id) AS Visits
FROM Customers
JOIN Vehicles ON Customers.customer_id = Vehicles.customer_id
JOIN Service_Records ON Vehicles.vehicle_id = Service_Records.vehicle_id
GROUP BY customer_name
HAVING COUNT(service_id) > 1;
```

customer_name	Visits
Arun	2
Priya	2
Rahul	2

Verification output of Query 7 with a condition present in the sample data

Query 8 : Display pending payments

```
SELECT customer_name, vehicle_number, total_amount
FROM Customers
JOIN Vehicles ON Customers.customer_id = Vehicles.customer_id
JOIN Service_Records ON Vehicles.vehicle_id = Service_Records.vehicle_id
JOIN Bills ON Service_Records.service_id = Bills.service_id
WHERE payment_status = 'Pending';
```

Concepts used: *Multiple INNER JOINs · WHERE filter*

Output

customer_name	vehicle_number	total_amount
Priya	KA05XY5678	2500.00
Arun	TS08PQ4321	2000.00
Karthik	MH12AA1111	1800.00

Output of Query 8 obtained from the live database

Query 9 : Find the most common service

```
SELECT service_type, COUNT(*) AS Frequency
FROM Service_Records
GROUP BY service_type
ORDER BY Frequency DESC
LIMIT 1;
```

Concepts used: *GROUP BY · COUNT() · ORDER BY · LIMIT*

Output

service_type	Frequency
General Service	3

Output of Query 9 obtained from the live database

Query 10 : Calculate monthly service revenue

```
SELECT MONTH(service_date) AS Month, SUM(total_amount) AS Revenue
FROM Service_Records
JOIN Bills ON Service_Records.service_id = Bills.service_id
GROUP BY MONTH(service_date);
```

Concepts used: *MONTH() · SUM() · GROUP BY*

Output

Month	Revenue
6	1800.00
7	29700.00

Output of Query 10 obtained from the live database

5. ADDITIONAL PRACTICE QUERIES WITH OUTPUT (QUERIES 11 – 20)

The following ten queries extend the analysis to further areas such as revenue distribution, customer loyalty, text filtering and analytical ranking. As before, every output is the actual result set returned by the database.

Query 11 : Find the highest bill

```
SELECT * FROM Bills
ORDER BY total_amount DESC
LIMIT 1;
```

Concepts used: *ORDER BY · LIMIT*

Output

bill_id	service_id	total_amount	payment_status
1	1	8000.00	Paid

Output of Query 11 obtained from the live database

Query 12 : Find the mechanic with the highest revenue

```
SELECT mechanic_name, SUM(total_amount) AS Revenue
FROM Mechanics
JOIN Service_Records ON Mechanics.mechanic_id = Service_Records.mechanic_id
JOIN Bills ON Service_Records.service_id = Bills.service_id
GROUP BY mechanic_name
ORDER BY Revenue DESC;
```

Concepts used: *INNER JOINs · SUM() · GROUP BY · ORDER BY*

Output

mechanic_name	Revenue
Ramesh	13000.00
Ganesh	7000.00
Mahesh	6700.00
Suresh	4800.00

Output of Query 12 obtained from the live database

Query 13 : Display customers who own more than one vehicle

```
SELECT customer_name, COUNT(vehicle_id) AS Vehicles
FROM Customers
JOIN Vehicles ON Customers.customer_id = Vehicles.customer_id
GROUP BY customer_name
HAVING COUNT(vehicle_id) > 1;
```

Concepts used: *GROUP BY · HAVING*

Output

(0 rows returned — the query executed successfully but no record in the current data satisfied the condition.)

> **Note:** *Every customer in the sample data owns exactly one vehicle, so no customer satisfies the condition of owning more than one vehicle and the result set is empty. The verification variant with the condition HAVING COUNT(vehicle_id) >= 1 is shown underneath.*

Verification run

```
SELECT customer_name, COUNT(vehicle_id) AS Vehicles
FROM Customers
JOIN Vehicles ON Customers.customer_id = Vehicles.customer_id
GROUP BY customer_name
HAVING COUNT(vehicle_id) >= 1;
```

customer_name	Vehicles
Arun	1
Karthik	1
Priya	1
Rahul	1
Sneha	1

Verification output of Query 13 with a condition present in the sample data

Query 14 : Find the average service cost

```
SELECT AVG(cost) AS Average_Cost
FROM Service_Records;
```

Concepts used: *AVG() aggregate function*

Output

Average_Cost
3937.500000

Output of Query 14 obtained from the live database

Query 15 : Find the most experienced mechanic

```
SELECT * FROM Mechanics
ORDER BY experience DESC
LIMIT 1;
```

Concepts used: *ORDER BY · LIMIT*

Output

mechanic_id	mechanic_name	specialization	experience
4	Ganesh	Painting	12

Output of Query 15 obtained from the live database

Query 16 : Display all engine repairs

```
SELECT * FROM Service_Records
WHERE service_type = 'Engine Repair';
```

Concepts used: *WHERE filter on text column*

Output

service_id	vehicle_id	mechanic_id	service_type	service_date	cost
1	1	1	Engine Repair	2026-07-10	8000.00
4	3	1	Engine Repair	2026-07-15	5000.00

Output of Query 16 obtained from the live database

Query 17 : Find customers from Chennai

```
SELECT * FROM Customers
WHERE city = 'Chennai';
```

Concepts used: *WHERE filter on text column*

Output

customer_id	customer_name	phone	city
1	Rahul	9876543210	Chennai

Output of Query 17 obtained from the live database

Query 18 : Find all services performed by a particular mechanic (Ramesh)

```
SELECT mechanic_name, service_type, service_date
FROM Mechanics
JOIN Service_Records ON Mechanics.mechanic_id = Service_Records.mechanic_id
WHERE mechanic_name = 'Ramesh';
```

Concepts used: *INNER JOIN · WHERE filter*

Output

mechanic_name	service_type	service_date
Ramesh	Engine Repair	2026-07-10

Ramesh	Engine Repair	2026-07-15

Output of Query 18 obtained from the live database

Query 19 : Display unpaid bill details with service information

```
SELECT bill_id, customer_name, vehicle_number, service_type, service_date, total_amount
FROM Bills
JOIN Service_Records ON Bills.service_id = Service_Records.service_id
JOIN Vehicles ON Service_Records.vehicle_id = Vehicles.vehicle_id
JOIN Customers ON Vehicles.customer_id = Customers.customer_id
WHERE payment_status = 'Pending';
```

Concepts used: *Four-table INNER JOIN chain · WHERE filter*

Output

bill_id	customer_name	vehicle_number	service_type	service_date	total_amount
3	Priya	KA05XY5678	General Service	2026-07-21	2500.00
6	Karthik	MH12AA1111	Electrical Repair	2026-06-30	1800.00
8	Arun	TS08PQ4321	General Service	2026-07-27	2000.00

Output of Query 19 obtained from the live database

Query 20 : Rank mechanics based on the number of services handled

```
SELECT mechanic_name, COUNT(service_id) AS Total_Services,
       RANK() OVER (ORDER BY COUNT(service_id) DESC) AS Service_Rank
FROM Mechanics
LEFT JOIN Service_Records ON Mechanics.mechanic_id = Service_Records.mechanic_id
GROUP BY mechanic_name;
```

Concepts used: *Window function RANK() OVER() · LEFT JOIN · GROUP BY*

Output

mechanic_name	Total_Services	Service_Rank
Ganesh	1	4
Mahesh	3	1
Ramesh	2	2
Suresh	2	2

Output of Query 20 obtained from the live database

6. RESULTS AND DISCUSSION

All twenty queries were executed successfully against the populated database and returned well-formed result sets. Sixteen queries produced one or more rows and four queries returned empty result sets, in each case because the sample data did not satisfy the condition being tested rather than because of any error. Verification variants were run for those four queries to confirm that the logic itself is correct.

6.1 Key Observations

- Customer-wise billing (Query 3) shows that Rahul accounts for the highest billing total of ₹11,000, followed by Priya with ₹9,500, which is consistent with the service records and bills stored in the database.
- Mechanic performance (Queries 2, 12 and 20) shows that Mahesh handled the highest number of services (three), while Ramesh generated the highest revenue of ₹13,000, because the two engine repairs assigned to him were the most expensive jobs. This illustrates that volume of work and revenue generated are different measures of performance.
- Pending payments (Queries 8 and 19) identify three bills awaiting settlement belonging to Priya, Karthik and Arun, with recoverable amounts of ₹2,500, ₹1,800 and ₹2,000 respectively. This is the most directly actionable report for the workshop owner.
- Monthly revenue (Query 10) shows that July 2026 produced ₹29,700 against ₹1,800 in June 2026, indicating a sharp increase in business activity.
- Service frequency (Queries 6 and 9) shows that General Service is the most common job with three occurrences, followed by Engine Repair and Electrical Repair with two each and Painting with one.
- The window function in Query 20 ranks Mahesh first, Ramesh and Suresh tied at second rank and Ganesh fourth. The tie demonstrates the difference between RANK() and ROW_NUMBER(), since RANK() assigns the same rank to equal values.

6.2 Significance of the Outputs

The outputs demonstrate how a small, well-normalised relational database can answer operational questions that would otherwise require manual computation from a register. Payment follow-up, customer billing, mechanic productivity and monthly revenue analysis are all produced by single SQL statements that execute instantaneously. Because the results are derived from the tables rather than stored separately, the reports always reflect the current state of the data and no separate report file has to be maintained.

6.3 SQL Features Demonstrated

Feature area	SQL constructs used
Data definition	CREATE DATABASE, CREATE TABLE, data types, PRIMARY KEY, FOREIGN KEY, AUTO_INCREMENT
Data manipulation	INSERT INTO … VALUES with multiple row tuples

Filtering	WHERE with equality conditions, date comparisons, IN and NOT IN
Joins	Two-table to four-table INNER JOIN chains; LEFT JOIN for inclusive counting
Aggregation	COUNT, SUM, AVG with GROUP BY
Group filtering	HAVING applied to aggregated counts
Subqueries	Non-correlated subquery used as an anti-join with NOT IN
Sorting and limiting	ORDER BY with DESC and LIMIT for top-N reporting
Date and time functions	CURDATE(), DATE_SUB() with INTERVAL, MONTH()
Window functions	RANK() OVER (ORDER BY …) for ranking without collapsing rows

Table 6.1 — Summary of SQL features demonstrated in the project

6.4 Testing and Validation

The complete script was executed end-to-end on a clean database instance. Every statement executed without error, and the row counts of all five tables matched the number of tuples inserted. The queries were then run individually and their result sets were verified against manual calculation of the sample data — for example, the customer-wise totals returned by Query 3 were cross-checked against the individual bills, and the mechanic ranking returned by Query 20 was compared with the service counts returned by Query 2. This testing confirms both the syntactic correctness of the SQL and the integrity of the underlying data.

7. CONCLUSION AND FUTURE SCOPE

7.1 Conclusion

This mini project designed, implemented and validated a relational database for a vehicle service centre using MySQL. A fully normalised schema consisting of five interrelated tables was created with primary keys, foreign keys and automatic surrogate identifiers, and was populated with representative sample data. Twenty SQL queries were then developed and executed to extract operationally useful information from the stored records.

The project demonstrates that the combination of a well-structured schema and standard SQL constructs is sufficient to deliver meaningful management reporting for a small business. Aggregation with GROUP BY and HAVING produced customer-wise billing, service frequency counts and pending payment lists; multi-table joins connected each bill back to the customer and vehicle it belongs to; subqueries identified vehicles with no recent service; and window functions ranked mechanics by workload. The normalised design eliminated redundancy while joins preserved complete information retrieval, and every query returned results consistent with the underlying data.

7.2 Future Scope

- Development of a graphical user interface or a web front end using PHP, Python or Java to make the system usable by non-technical staff.
- Addition of an inventory and spare-parts module with stock levels and consumption tracking linked to each service record.
- Introduction of role-based authentication to separate the access rights of administrators, service advisors and mechanics.
- Generation of printable PDF invoices and automated e-mail or SMS intimation to customers when a vehicle is ready or when a payment is overdue.
- Implementation of stored procedures, triggers and views to encapsulate frequently used reports and enforce business rules at the database level.
- Addition of appointment scheduling, service reminders based on the last service date and customer feedback collection.
- Migration to a cloud-hosted database with backup and recovery policies for multi-branch operation.

APPENDIX A — COMPLETE SQL SCRIPT

The following is the complete, ready-to-run SQL script for the project. It can be executed in a single pass from the MySQL command-line client as mysql -u root -p < vehicle_service_center.sql or by opening the file vehicle_service_center.sql in MySQL Workbench and executing it.

```
--  MySQL MINI PROJECT  :  VEHICLE SERVICE CENTER MANAGEMENT SYSTEM
--  ------------------------------------------------------------------------
--  Description : This project simulates a vehicle service center where
--                customers bring their vehicles for servicing. The database
--                stores customer details, vehicle information, mechanics,
--                services performed and billing details.
--
--  Tables      : Customers, Vehicles, Mechanics, Service_Records, Bills
--  Queries     : 20 practice queries (SELECT, JOIN, GROUP BY, HAVING,
--                subqueries, aggregate and window functions)
--
--  Tested on   : MySQL 8.x  /  MariaDB 10.4+
--  Usage       : mysql -u root -p < vehicle_service_center.sql

DROP DATABASE IF EXISTS vehicle_service_center;
CREATE DATABASE vehicle_service_center;
USE vehicle_service_center;

--  SECTION 1 : TABLE CREATION (DDL)

-- ------------------------------------------------------------------------
-- Table 1 : Customers  -  stores customer information
-- ------------------------------------------------------------------------
CREATE TABLE Customers (
    customer_id   INT PRIMARY KEY AUTO_INCREMENT,
    customer_name VARCHAR(50),
    phone         VARCHAR(15),
    city          VARCHAR(30)
);

-- ------------------------------------------------------------------------
-- Table 2 : Vehicles  -  each customer can own multiple vehicles
-- ------------------------------------------------------------------------
CREATE TABLE Vehicles (
    vehicle_id     INT PRIMARY KEY AUTO_INCREMENT,
    customer_id    INT,
    vehicle_number VARCHAR(15),
    vehicle_model  VARCHAR(50),
    vehicle_type   VARCHAR(30),
    FOREIGN KEY (customer_id) REFERENCES Customers(customer_id)
);

-- ------------------------------------------------------------------------
-- Table 3 : Mechanics  -  stores mechanic details
-- ------------------------------------------------------------------------
CREATE TABLE Mechanics (
    mechanic_id     INT PRIMARY KEY AUTO_INCREMENT,
    mechanic_name   VARCHAR(50),
    specialization  VARCHAR(50),
    experience      INT
);

-- ------------------------------------------------------------------------
-- Table 4 : Service_Records  -  stores every service performed
-- ------------------------------------------------------------------------
CREATE TABLE Service_Records (
    service_id   INT PRIMARY KEY AUTO_INCREMENT,
    vehicle_id   INT,
    mechanic_id  INT,
```

```
    service_type VARCHAR(50),
    service_date DATE,
    cost         DECIMAL(10,2),
    FOREIGN KEY (vehicle_id)  REFERENCES Vehicles(vehicle_id),
    FOREIGN KEY (mechanic_id) REFERENCES Mechanics(mechanic_id)
);

-- ------------------------------------------------------------------------
-- Table 5 : Bills  -  stores billing information
-- ------------------------------------------------------------------------
CREATE TABLE Bills (
    bill_id        INT PRIMARY KEY AUTO_INCREMENT,
    service_id     INT,
    total_amount   DECIMAL(10,2),
    payment_status VARCHAR(20),
    FOREIGN KEY (service_id) REFERENCES Service_Records(service_id)
);

--  SECTION 2 : SAMPLE DATA (DML)

-- Table 1 : Customers
INSERT INTO Customers (customer_name, phone, city)
VALUES
('Rahul',   '9876543210', 'Chennai'),
('Priya',   '9876543211', 'Bangalore'),
('Arun',    '9876543212', 'Hyderabad'),
('Sneha',   '9876543213', 'Coimbatore'),
('Karthik', '9876543214', 'Mumbai');

-- Table 2 : Vehicles
INSERT INTO Vehicles (customer_id, vehicle_number, vehicle_model, vehicle_type)
VALUES
(1, 'TN10AB1234', 'Hyundai i20',      'Car'),
(2, 'KA05XY5678', 'Honda City',       'Car'),
(3, 'TS08PQ4321', 'Royal Enfield',    'Bike'),
(4, 'TN22KL9090', 'Maruti Swift',     'Car'),
(5, 'MH12AA1111', 'TVS Apache',       'Bike');

-- Table 3 : Mechanics
INSERT INTO Mechanics (mechanic_name, specialization, experience)
VALUES
('Ramesh', 'Engine',          10),
('Suresh', 'Electrical',      8),
('Mahesh', 'General Service',  6),
('Ganesh', 'Painting',        12);

-- Table 4 : Service_Records
INSERT INTO Service_Records (vehicle_id, mechanic_id, service_type, service_date, cost)
VALUES
(1, 1, 'Engine Repair',     '2026-07-10', 8000),
(1, 2, 'Electrical Repair', '2026-07-20', 3000),
(2, 3, 'General Service',   '2026-07-21', 2500),
(3, 1, 'Engine Repair',     '2026-07-15', 5000),
(4, 3, 'General Service',   '2026-07-18', 2200),
(5, 2, 'Electrical Repair', '2026-06-30', 1800),
(2, 4, 'Painting',          '2026-07-25', 7000),
(3, 3, 'General Service',   '2026-07-27', 2000);

-- Table 5 : Bills
INSERT INTO Bills (service_id, total_amount, payment_status)
VALUES
(1, 8000, 'Paid'),
(2, 3000, 'Paid'),
(3, 2500, 'Pending'),
(4, 5000, 'Paid'),
(5, 2200, 'Paid'),
(6, 1800, 'Pending'),
```

```
(7, 7000, 'Paid'),
(8, 2000, 'Pending');

--  SECTION 3 : PRACTICE QUERIES (1 - 10)

-- ------------------------------------------------------------------------
-- Query 1 : Display vehicles serviced today.
-- Concepts: INNER JOIN, CURDATE()
-- ------------------------------------------------------------------------
SELECT
    vehicle_number,
    vehicle_model,
    service_type,
    service_date
FROM Vehicles
JOIN Service_Records
  ON Vehicles.vehicle_id = Service_Records.vehicle_id
WHERE service_date = CURDATE();
-- NOTE: The sample data contains services from July 2026, so this query returns
--       0 rows on any other date. For demonstration use a date present in the
--       data, e.g.:  WHERE service_date = '2026-07-10';

-- ------------------------------------------------------------------------
-- Query 2 : Find the mechanic handling the most services.
-- Concepts: GROUP BY, COUNT(), ORDER BY, LIMIT
-- ------------------------------------------------------------------------
SELECT
    mechanic_name,
    COUNT(service_id) AS Total_Services
FROM Mechanics
JOIN Service_Records
  ON Mechanics.mechanic_id = Service_Records.mechanic_id
GROUP BY mechanic_name
ORDER BY Total_Services DESC
LIMIT 1;

-- ------------------------------------------------------------------------
-- Query 3 : Calculate customer-wise bill amount.
-- Concepts: Multiple INNER JOINs, SUM(), GROUP BY
-- ------------------------------------------------------------------------
SELECT
    customer_name,
    SUM(total_amount) AS Total_Bill
FROM Customers
JOIN Vehicles
  ON Customers.customer_id = Vehicles.customer_id
JOIN Service_Records
  ON Vehicles.vehicle_id = Service_Records.vehicle_id
JOIN Bills
  ON Service_Records.service_id = Bills.service_id
GROUP BY customer_name;

-- ------------------------------------------------------------------------
-- Query 4 : Find vehicles not serviced in the last year.
-- Concepts: Subquery (NOT IN), DATE_SUB()
-- ------------------------------------------------------------------------
SELECT
    vehicle_number,
    vehicle_model
FROM Vehicles
WHERE vehicle_id NOT IN
(
    SELECT vehicle_id
    FROM Service_Records
    WHERE service_date >= DATE_SUB(CURDATE(), INTERVAL 1 YEAR)
);
-- NOTE: All sample services fall within one year of the current date, so every
```

```
--       vehicle qualifies as "serviced recently" and 0 rows are returned.
--       To see the subquery in action, narrow the window, e.g.:
--       WHERE service_date >= DATE_SUB('2026-08-01', INTERVAL 1 MONTH);

-- ------------------------------------------------------------------------
-- Query 5 : Display service history for a vehicle (example: TN10AB1234).
-- Concepts: INNER JOIN, WHERE filter, ORDER BY
-- ------------------------------------------------------------------------
SELECT
    vehicle_number,
    service_type,
    service_date,
    cost
FROM Vehicles
JOIN Service_Records
  ON Vehicles.vehicle_id = Service_Records.vehicle_id
WHERE vehicle_number = 'TN10AB1234'
ORDER BY service_date;

-- ------------------------------------------------------------------------
-- Query 6 : Count services by type.
-- Concepts: GROUP BY, COUNT(*)
-- ------------------------------------------------------------------------
SELECT
    service_type,
    COUNT(*) AS Total
FROM Service_Records
GROUP BY service_type;

-- ------------------------------------------------------------------------
-- Query 7 : Find customers with more than three visits.
-- Concepts: HAVING
-- ------------------------------------------------------------------------
SELECT
    customer_name,
    COUNT(service_id) AS Visits
FROM Customers
JOIN Vehicles
  ON Customers.customer_id = Vehicles.customer_id
JOIN Service_Records
  ON Vehicles.vehicle_id = Service_Records.vehicle_id
GROUP BY customer_name
HAVING COUNT(service_id) > 3;
-- NOTE: In the sample data no customer exceeds 3 visits, so 0 rows are
--       returned. Replacing the condition with  HAVING COUNT(service_id) > 1
--       lists the customers who visited more than once.

-- ------------------------------------------------------------------------
-- Query 8 : Display pending payments.
-- Concepts: Multiple INNER JOINs, WHERE filter
-- ------------------------------------------------------------------------
SELECT
    customer_name,
    vehicle_number,
    total_amount
FROM Customers
JOIN Vehicles
  ON Customers.customer_id = Vehicles.customer_id
JOIN Service_Records
  ON Vehicles.vehicle_id = Service_Records.vehicle_id
JOIN Bills
  ON Service_Records.service_id = Bills.service_id
WHERE payment_status = 'Pending';

-- ------------------------------------------------------------------------
-- Query 9 : Find the most common service.
-- Concepts: GROUP BY, COUNT(), ORDER BY, LIMIT
```

```
-- ------------------------------------------------------------------------
SELECT
    service_type,
    COUNT(*) AS Frequency
FROM Service_Records
GROUP BY service_type
ORDER BY Frequency DESC
LIMIT 1;

-- ------------------------------------------------------------------------
-- Query 10 : Calculate monthly service revenue.
-- Concepts: MONTH(), SUM(), GROUP BY
-- ------------------------------------------------------------------------
SELECT
    MONTH(service_date) AS Month,
    SUM(total_amount)   AS Revenue
FROM Service_Records
JOIN Bills
  ON Service_Records.service_id = Bills.service_id
GROUP BY MONTH(service_date);

--  SECTION 4 : ADDITIONAL PRACTICE QUERIES (11 - 20)

-- ------------------------------------------------------------------------
-- Query 11 : Find the highest bill.
-- Concepts: ORDER BY, LIMIT
-- ------------------------------------------------------------------------
SELECT *
FROM Bills
ORDER BY total_amount DESC
LIMIT 1;

-- ------------------------------------------------------------------------
-- Query 12 : Find the mechanic with the highest revenue.
-- Concepts: INNER JOINs, SUM(), GROUP BY, ORDER BY
-- ------------------------------------------------------------------------
SELECT
    mechanic_name,
    SUM(total_amount) AS Revenue
FROM Mechanics
JOIN Service_Records
  ON Mechanics.mechanic_id = Service_Records.mechanic_id
JOIN Bills
  ON Service_Records.service_id = Bills.service_id
GROUP BY mechanic_name
ORDER BY Revenue DESC;

-- ------------------------------------------------------------------------
-- Query 13 : Display customers who own more than one vehicle.
-- Concepts: GROUP BY, HAVING
-- ------------------------------------------------------------------------
SELECT
    customer_name,
    COUNT(vehicle_id) AS Vehicles
FROM Customers
JOIN Vehicles
  ON Customers.customer_id = Vehicles.customer_id
GROUP BY customer_name
HAVING COUNT(vehicle_id) > 1;
-- NOTE: Every customer in the sample data owns exactly one vehicle, so 0 rows
--       are returned. Lowering the threshold to  >= 1  lists all customers.

-- ------------------------------------------------------------------------
-- Query 14 : Find the average service cost.
-- Concepts: AVG() aggregate function
-- ------------------------------------------------------------------------
SELECT
```

```
    AVG(cost) AS Average_Cost
FROM Service_Records;

-- ------------------------------------------------------------------------
-- Query 15 : Find the most experienced mechanic.
-- Concepts: ORDER BY, LIMIT
-- ------------------------------------------------------------------------
SELECT *
FROM Mechanics
ORDER BY experience DESC
LIMIT 1;

-- ------------------------------------------------------------------------
-- Query 16 : Display all engine repairs.
-- Concepts: WHERE filter on text column
-- ------------------------------------------------------------------------
SELECT *
FROM Service_Records
WHERE service_type = 'Engine Repair';

-- ------------------------------------------------------------------------
-- Query 17 : Find customers from Chennai.
-- Concepts: WHERE filter on text column
-- ------------------------------------------------------------------------
SELECT *
FROM Customers
WHERE city = 'Chennai';

-- ------------------------------------------------------------------------
-- Query 18 : Find all services performed by a particular mechanic (Ramesh).
-- Concepts: INNER JOIN, WHERE filter
-- ------------------------------------------------------------------------
SELECT
    mechanic_name,
    service_type,
    service_date
FROM Mechanics
JOIN Service_Records
  ON Mechanics.mechanic_id = Service_Records.mechanic_id
WHERE mechanic_name = 'Ramesh';

-- ------------------------------------------------------------------------
-- Query 19 : Display unpaid bill details with service information.
-- Concepts: Four-table INNER JOIN chain, WHERE filter
-- ------------------------------------------------------------------------
SELECT
    bill_id,
    customer_name,
    vehicle_number,
    service_type,
    service_date,
    total_amount
FROM Bills
JOIN Service_Records
  ON Bills.service_id = Service_Records.service_id
JOIN Vehicles
  ON Service_Records.vehicle_id = Vehicles.vehicle_id
JOIN Customers
  ON Vehicles.customer_id = Customers.customer_id
WHERE payment_status = 'Pending';

-- ------------------------------------------------------------------------
-- Query 20 : Rank mechanics based on the number of services handled.
-- Concepts: Window function RANK() OVER(), LEFT JOIN, GROUP BY
-- ------------------------------------------------------------------------
SELECT
    mechanic_name,
```

```
    COUNT(service_id) AS Total_Services,
    RANK() OVER (ORDER BY COUNT(service_id) DESC) AS Service_Rank
FROM Mechanics
LEFT JOIN Service_Records
  ON Mechanics.mechanic_id = Service_Records.mechanic_id
GROUP BY mechanic_name;

--  END OF SCRIPT
```

APPENDIX B — QUERY OUTPUT LOG

The output log below is the raw session transcript captured while executing all twenty queries and the four verification runs against the live database. It is reproduced here as evidence of successful execution.

```
VEHICLE SERVICE CENTER MANAGEMENT SYSTEM - QUERY OUTPUT LOG
Generated from live MySQL/MariaDB session - today = 2026-10-13

================================================================================
========
QUERY 1 : Display vehicles serviced today
================================================================================
========
SELECT vehicle_number, vehicle_model, service_type, service_date
FROM Vehicles
JOIN Service_Records ON Vehicles.vehicle_id = Service_Records.vehicle_id
WHERE service_date = CURDATE();

(0 rows returned)

================================================================================
========
QUERY 2 : Find the mechanic handling the most services
================================================================================
========
SELECT mechanic_name, COUNT(service_id) AS Total_Services
FROM Mechanics
JOIN Service_Records ON Mechanics.mechanic_id = Service_Records.mechanic_id
GROUP BY mechanic_name
ORDER BY Total_Services DESC
LIMIT 1;

+---------------+----------------+
| mechanic_name | Total_Services |
+---------------+----------------+
| Mahesh        | 3              |
+---------------+----------------+

================================================================================
========
QUERY 3 : Calculate customer-wise bill amount
================================================================================
========
SELECT customer_name, SUM(total_amount) AS Total_Bill
FROM Customers
JOIN Vehicles ON Customers.customer_id = Vehicles.customer_id
JOIN Service_Records ON Vehicles.vehicle_id = Service_Records.vehicle_id
JOIN Bills ON Service_Records.service_id = Bills.service_id
GROUP BY customer_name;

+---------------+------------+
| customer_name | Total_Bill |
+---------------+------------+
| Arun          | 7000.00    |
| Karthik       | 1800.00    |
| Priya         | 9500.00    |
| Rahul         | 11000.00   |
| Sneha         | 2200.00    |
+---------------+------------+

================================================================================
========
QUERY 4 : Find vehicles not serviced in the last year
================================================================================
========
SELECT vehicle_number, vehicle_model
FROM Vehicles
```

```
WHERE vehicle_id NOT IN
(
   SELECT vehicle_id
   FROM Service_Records
   WHERE service_date >= DATE_SUB(CURDATE(), INTERVAL 1 YEAR)
);

(0 rows returned)

================================================================================
========
QUERY 5 : Display service history for a vehicle (TN10AB1234)
================================================================================
========
SELECT vehicle_number, service_type, service_date, cost
FROM Vehicles
JOIN Service_Records ON Vehicles.vehicle_id = Service_Records.vehicle_id
WHERE vehicle_number = 'TN10AB1234'
ORDER BY service_date;

+---------------+------------------+--------------+---------+
| vehicle_number | service_type      | service_date | cost    |
+---------------+------------------+--------------+---------+
| TN10AB1234     | Engine Repair     | 2026-07-10   | 8000.00 |
| TN10AB1234     | Electrical Repair | 2026-07-20   | 3000.00 |
+---------------+------------------+--------------+---------+

================================================================================
========
QUERY 6 : Count services by type
================================================================================
========
SELECT service_type, COUNT(*) AS Total
FROM Service_Records
GROUP BY service_type;

+------------------+-------+
| service_type      | Total |
+------------------+-------+
| Electrical Repair | 2     |
| Engine Repair     | 2     |
| General Service   | 3     |
| Painting          | 1     |
+------------------+-------+

================================================================================
========
QUERY 7 : Find customers with more than three visits
================================================================================
========
SELECT customer_name, COUNT(service_id) AS Visits
FROM Customers
JOIN Vehicles ON Customers.customer_id = Vehicles.customer_id
JOIN Service_Records ON Vehicles.vehicle_id = Service_Records.vehicle_id
GROUP BY customer_name
HAVING COUNT(service_id) > 3;

(0 rows returned)

================================================================================
========
QUERY 8 : Display pending payments
================================================================================
========
SELECT customer_name, vehicle_number, total_amount
FROM Customers
JOIN Vehicles ON Customers.customer_id = Vehicles.customer_id
JOIN Service_Records ON Vehicles.vehicle_id = Service_Records.vehicle_id
JOIN Bills ON Service_Records.service_id = Bills.service_id
WHERE payment_status = 'Pending';
```

```
+--------------+---------------+-------------+
| customer_name | vehicle_number | total_amount |
+--------------+---------------+-------------+
| Priya         | KA05XY5678     | 2500.00      |
| Arun          | TS08PQ4321     | 2000.00      |
| Karthik       | MH12AA1111     | 1800.00      |
+--------------+---------------+-------------+

================================================================================
========
QUERY 9 : Find the most common service
================================================================================
========
SELECT service_type, COUNT(*) AS Frequency
FROM Service_Records
GROUP BY service_type
ORDER BY Frequency DESC
LIMIT 1;

+----------------+-----------+
| service_type    | Frequency |
+----------------+-----------+
| General Service | 3         |
+----------------+-----------+

================================================================================
========
QUERY 10 : Calculate monthly service revenue
================================================================================
========
SELECT MONTH(service_date) AS Month, SUM(total_amount) AS Revenue
FROM Service_Records
JOIN Bills ON Service_Records.service_id = Bills.service_id
GROUP BY MONTH(service_date);

+-------+----------+
| Month | Revenue  |
+-------+----------+
| 6     | 1800.00  |
| 7     | 29700.00 |
+-------+----------+

================================================================================
========
QUERY 11 : Find the highest bill
================================================================================
========
SELECT * FROM Bills
ORDER BY total_amount DESC
LIMIT 1;

+---------+------------+--------------+----------------+
| bill_id | service_id | total_amount | payment_status |
+---------+------------+--------------+----------------+
| 1       | 1          | 8000.00      | Paid           |
+---------+------------+--------------+----------------+

================================================================================
========
QUERY 12 : Find the mechanic with the highest revenue
================================================================================
========
SELECT mechanic_name, SUM(total_amount) AS Revenue
FROM Mechanics
JOIN Service_Records ON Mechanics.mechanic_id = Service_Records.mechanic_id
JOIN Bills ON Service_Records.service_id = Bills.service_id
GROUP BY mechanic_name
ORDER BY Revenue DESC;

+---------------+----------+
| mechanic_name | Revenue  |
```

```
+--------------+----------+
| Ramesh       | 13000.00 |
| Ganesh       | 7000.00  |
| Mahesh       | 6700.00  |
| Suresh       | 4800.00  |
+--------------+----------+

================================================================================
========
QUERY 13 : Display customers who own more than one vehicle
================================================================================
========
SELECT customer_name, COUNT(vehicle_id) AS Vehicles
FROM Customers
JOIN Vehicles ON Customers.customer_id = Vehicles.customer_id
GROUP BY customer_name
HAVING COUNT(vehicle_id) > 1;

(0 rows returned)

================================================================================
========
QUERY 14 : Find the average service cost
================================================================================
========
SELECT AVG(cost) AS Average_Cost
FROM Service_Records;

+--------------+
| Average_Cost |
+--------------+
| 3937.500000  |
+--------------+

================================================================================
========
QUERY 15 : Find the most experienced mechanic
================================================================================
========
SELECT * FROM Mechanics
ORDER BY experience DESC
LIMIT 1;

+-------------+---------------+----------------+------------+
| mechanic_id | mechanic_name | specialization | experience |
+-------------+---------------+----------------+------------+
| 4           | Ganesh        | Painting       | 12         |
+-------------+---------------+----------------+------------+

================================================================================
========
QUERY 16 : Display all engine repairs
================================================================================
========
SELECT * FROM Service_Records
WHERE service_type = 'Engine Repair';

+------------+------------+------------+--------------+--------------+---------+
| service_id | vehicle_id | mechanic_id | service_type  | service_date | cost    |
+------------+------------+------------+--------------+--------------+---------+
| 1          | 1          | 1          | Engine Repair | 2026-07-10   | 8000.00 |
| 4          | 3          | 1          | Engine Repair | 2026-07-15   | 5000.00 |
+------------+------------+------------+--------------+--------------+---------+

================================================================================
========
QUERY 17 : Find customers from Chennai
================================================================================
========
SELECT * FROM Customers
WHERE city = 'Chennai';
```

```
+------------+--------------+-----------+--------+
| customer_id | customer_name | phone      | city    |
+------------+--------------+-----------+--------+
| 1          | Rahul        | 9876543210 | Chennai |
+------------+--------------+-----------+--------+

================================================================================
========
QUERY 18 : Find all services performed by a particular mechanic (Ramesh)
================================================================================
========
SELECT mechanic_name, service_type, service_date
FROM Mechanics
JOIN Service_Records ON Mechanics.mechanic_id = Service_Records.mechanic_id
WHERE mechanic_name = 'Ramesh';

+--------------+--------------+-------------+
| mechanic_name | service_type  | service_date |
+--------------+--------------+-------------+
| Ramesh       | Engine Repair | 2026-07-10   |
| Ramesh       | Engine Repair | 2026-07-15   |
+--------------+--------------+-------------+

================================================================================
========
QUERY 19 : Display unpaid bill details with service information
================================================================================
========
SELECT bill_id, customer_name, vehicle_number, service_type, service_date, total_amount
FROM Bills
JOIN Service_Records ON Bills.service_id = Service_Records.service_id
JOIN Vehicles ON Service_Records.vehicle_id = Vehicles.vehicle_id
JOIN Customers ON Vehicles.customer_id = Customers.customer_id
WHERE payment_status = 'Pending';

+---------+--------------+---------------+------------------+-------------+-------------+
| bill_id | customer_name | vehicle_number | service_type      | service_date | total_amount |
+---------+--------------+---------------+------------------+-------------+-------------+
| 3       | Priya        | KA05XY5678     | General Service   | 2026-07-21   | 2500.00      |
| 6       | Karthik      | MH12AA1111     | Electrical Repair | 2026-06-30   | 1800.00      |
| 8       | Arun         | TS08PQ4321     | General Service   | 2026-07-27   | 2000.00      |
+---------+--------------+---------------+------------------+-------------+-------------+

================================================================================
========
QUERY 20 : Rank mechanics based on the number of services handled
================================================================================
========
SELECT mechanic_name, COUNT(service_id) AS Total_Services,
       RANK() OVER (ORDER BY COUNT(service_id) DESC) AS Service_Rank
FROM Mechanics
LEFT JOIN Service_Records ON Mechanics.mechanic_id = Service_Records.mechanic_id
GROUP BY mechanic_name;

+--------------+---------------+-------------+
| mechanic_name | Total_Services | Service_Rank |
+--------------+---------------+-------------+
| Ganesh       | 1             | 4           |
| Mahesh       | 3             | 1           |
| Ramesh       | 2             | 2           |
| Suresh       | 2             | 2           |
+--------------+---------------+-------------+

--------------------------------------------------------------------------------
VERIFICATION : Query 1 with a date present in the data
--------------------------------------------------------------------------------
SELECT vehicle_number, vehicle_model, service_type, service_date
FROM Vehicles
JOIN Service_Records ON Vehicles.vehicle_id = Service_Records.vehicle_id
WHERE service_date = '2026-07-10';
```

```
+---------------+--------------+--------------+-------------+
| vehicle_number | vehicle_model | service_type  | service_date |
+---------------+--------------+--------------+-------------+
| TN10AB1234     | Hyundai i20   | Engine Repair | 2026-07-10   |
+---------------+--------------+--------------+-------------+

--------------------------------------------------------------------------------
VERIFICATION : Query 4 with a narrower one-month window
--------------------------------------------------------------------------------
SELECT vehicle_number, vehicle_model
FROM Vehicles
WHERE vehicle_id NOT IN
(
    SELECT vehicle_id
    FROM Service_Records
    WHERE service_date >= DATE_SUB('2026-08-01', INTERVAL 1 MONTH)
);

+---------------+--------------+
| vehicle_number | vehicle_model |
+---------------+--------------+
| MH12AA1111     | TVS Apache    |
+---------------+--------------+

--------------------------------------------------------------------------------
VERIFICATION : Query 7 with threshold > 1 visit
--------------------------------------------------------------------------------
SELECT customer_name, COUNT(service_id) AS Visits
FROM Customers
JOIN Vehicles ON Customers.customer_id = Vehicles.customer_id
JOIN Service_Records ON Vehicles.vehicle_id = Service_Records.vehicle_id
GROUP BY customer_name
HAVING COUNT(service_id) > 1;

+--------------+--------+
| customer_name | Visits |
+--------------+--------+
| Arun          | 2      |
| Priya         | 2      |
| Rahul         | 2      |
+--------------+--------+

--------------------------------------------------------------------------------
VERIFICATION : Query 13 with threshold >= 1 vehicle
--------------------------------------------------------------------------------
SELECT customer_name, COUNT(vehicle_id) AS Vehicles
FROM Customers
JOIN Vehicles ON Customers.customer_id = Vehicles.customer_id
GROUP BY customer_name
HAVING COUNT(vehicle_id) >= 1;

+--------------+----------+
| customer_name | Vehicles |
+--------------+----------+
| Arun          | 1        |
| Karthik       | 1        |
| Priya         | 1        |
| Rahul         | 1        |
| Sneha         | 1        |
+--------------+----------+
```